Final Exam - Robotic Technology

Question 1 – Localization and Map Handling

Objective

Configure and run **map-based localization** using **Nav2 AMCL** and **map_server**.

Tasks

1. Load a provided occupancy grid map using `nav2_map_server`.
2. Configure and launch `nav2_amcl` for localization.
3. Ensure the robot can:
    - Publish its estimated pose in the map frame
    - Maintain a valid `map → odom → base_link` TF tree

4. Demonstrate that localization converges by:
    - Visualizing the pose estimate in RViz
    - Showing particle cloud convergence

Requirements

- Correct YAML configuration for:
    - Map server
    - AMCL parameters
- Proper ROS 2 launch file
- TF frames must be correct and consistent

Deliverables

- Launch file
- Parameter YAML files
- Screenshot or short recording of RViz showing successful localization

Question 2 – Global Path Planning Server (A*)

Objective

Implement an **A*** global planner as a **ROS 2 node acting as a service**.

Tasks

1. Implement a ROS 2 **service server** that:
    - Subscribes to the robot's current pose in the `map` frame
    - Receives a goal pose (`geometry_msgs/PoseStamped`)
2. Use the occupancy grid from the map server to:
    - Construct a grid-based graph
    - Perform **A*** path planning
3. Output the computed path as:
    - `nav_msgs/Path`

Requirements

- Correct A* implementation:
    - Cost function
    - Heuristic (e.g., Euclidean or Manhattan distance)

- Collision-free path
- Proper frame usage (`map` frame)

Deliverables

- Source code of the planner node
- Service definition (if custom)
- Visualization of the path in RViz

Question 3 – Reinforcement Learning for Line Following

Objective

Design and train a **Reinforcement Learning (RL)** controller for line following.

Tasks

1. Define an RL problem where:
    - The robot must follow the trajectory that is given by your path planner.

2. Implement and train an RL algorithm:
    - Examples: Q-learning, DQN, DDPG

3. Integrate the trained policy into a ROS 2 node controlling the robot

Requirements

- Clear definition of:
    - State space (observation)
    - Action space
    - Reward function (design your own reward function and explain why you choose it)
- Use the designed environment and reward function to train a policy using an appropriate RL algorithm. (what discussed in class don't use any open source RL packages like stable-baseline)

Deliverables

- Description of the RL formulation
- Training code
- ROS 2 control node
- Video or simulation demonstration

Bonus Question – Classical Control for Line Following

Objective

Implement a **classical control approach** for line following and compare performance.

Tasks

1. Implement a classical controller:
    - PID
    - MPC (more points that others)
2. Control the robot to follow the same line as in Question 3